Testing the Reliability of AI Image Detectors Using Adversarial Perturbations

Joyce Yang Yue En[1], Wei Mingyang[2], Daniel Lee Jek Han[3]
[1]Methodist Girls' School, 11 Blackmore Drive, Singapore 599986
[2]Hwa Chong Institution (High School), 661 Bukit Timah Rd, Singapore 269734
[3]Defence Science and Technology Agency, 1 Depot Road, Defence Technology Tower A, Singapore 109679

1. ABSTRACT

The present study evaluates the effectiveness of existing open-source AI adversarial image attackers. This was done by selecting an AI image detector, a spectral AI-generated detector (SPAI) that represents typical models, then testing how it would perform against different types of added random noise such as Gaussian noise and salt and pepper noise and comparing them with adversarial attackers such as AnyAttack and Diff-PGD, with different adjusted settings such as the epsilon (EPS) and number of iteration steps to find how it would hinder the ability of SPAI to classify images correctly. The goal was to find a model with settings where the adversarial image produced by the attacker would contain enough adversarial noise to fool the detector while being minimally distinguished by eye. After various tests, we concluded that Diff-PGD was able to generate adversarial images visually similar to the original image while still greatly decreasing the accuracy of the AI detector.

2. INTRODUCTION

2.1 - Background

In today's volatile and complex world, the reliability of AI-generated media detection has become increasingly important, as advancements in generative models continue to blur the line between authentic and fabricated imagery. Given the high volume of daily usage of AI image generation [1], it has therefore become increasingly necessary to adopt advanced solutions that can reliably distinguish these images to safeguard digital integrity.

2.2 - AI Forensic Image Systems

Currently, there are many existing image detectors, though most of them are prone to adversarial attacks such as addition or removal of noise, manual addition of elements into images, or alteration of their resolution, which greatly affect their ability to accurately detect AI-generated images. [2]

Our project aims to contribute to a deeper understanding of adversarial vulnerabilities in forensic AI detectors, highlight the need for more robust detection mechanisms, and provide some solutions to tackle the issues of AI-generated images.

3. MATERIALS AND METHODS

3.1 - Adversarial Attacks

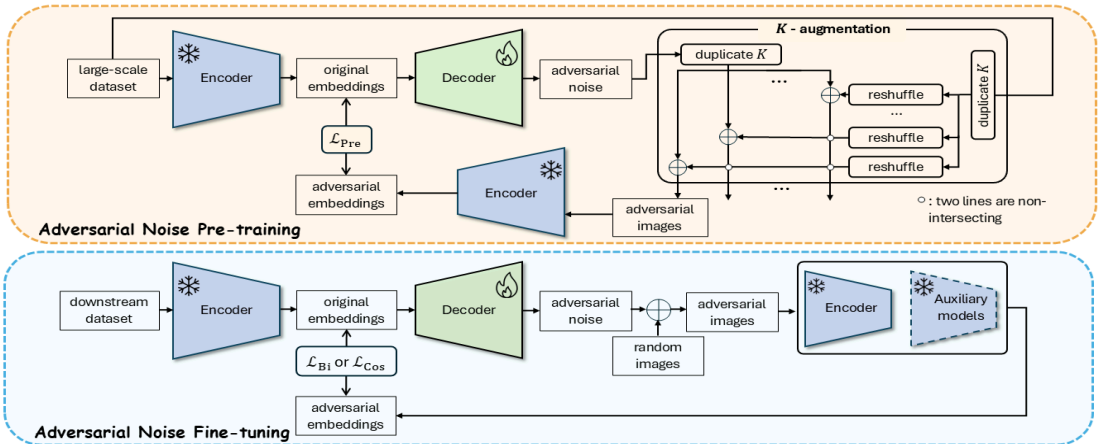
We selected two adversarial attackers that represent different but complementary approaches to adversarial example generation in order to uncover the potential flaws in forensic AI image-analysis systems. The first attacker is AnyAttack, a self-supervised adversarial noise generator capable of targeting modern vision-language models — a class of artificial intelligence models designed to understand and generate information by jointly processing visual and textual data [3] — while evading forensic detectors. The second attacker is

Diff-PGD, a diffusion-based variant of projected gradient descent designed to produce highly realistic perturbations. These two methods allow us to evaluate forensic systems under both traditional gradient-based perturbations.

3.1.1 - AnyAttack

AnyAttack [4] represents a more modern class of self-supervised attackers. Rather than relying on labels or direct gradient access to the target model, AnyAttack uses a pre-trained adversarial noise generator to craft perturbations that induce targeted outputs. This scalability allows AnyAttack to exploit vulnerabilities that emerge from multimodal alignment (such as image–text reasoning), and is increasingly relevant in our modern world as modern forensic systems start integrating VLMs into their analysis pipelines.

Figure 1: How AnyAttack works



For Adversarial Noise Pre-training, the process steps are as follows:

1. A decoder network F is trained to produce adversarial noise δ while using a frozen encoder E.
2. When given a batch of images, it extracts their embeddings using the encoder E
3. These embeddings are fed into the decoder F, which generates adversarial noise δ
4. Use a K-augmentation strategy that creates multiple shuffled versions of the original images within each mini-batch to improve efficiency of the attacks
5. The adversarial noise δ is added to the original images to produce a new image.

For Adversarial Noise Fine-Tuning, the pre-trained decoder F is adapted to a specific downstream dataset. Then, an unrelated random image x is provided from an external dataset as the clean image for the decoder to generate adversarial noise specific to the target task and add this noise to the clean image (x + δ). For a detailed process of noise generation in AnyAttack, refer to Appendix 8.1.1.

The advantage of this adversarial attacker is that it operates in a fully self-supervised manner, requiring no human guidance. This is because the encoder is frozen (its parameters do not update during training) and hence it does not require target labels as supervision throughout the training process. This enables us to observe how an unsupervised AI system would naturally generate adversarial perturbations.

This design also gives AnyAttack exceptional flexibility, since it can convert any image into an adversarial example that produces an attacker-specified output, enabling it to both target VLMs and evade forensic detectors by causing them to classify the manipulated image as

genuine. Therefore, we decided to use this adversarial attacker in our experiments to evade forensic detection.

3.1.2 - Diffusion Projected Gradient Descent (Diff-PGD)

PGD (Projected Gradient Descent) is a white-box, gradient-based adversarial attack method that iteratively adjusts an image in the direction that maximizes model error, while "projecting" the perturbation back into a valid constraint set. Diff-PGD [5] modifies this process by integrating a diffusion model into the optimisation loop, so the adversarial perturbations stay close to the natural image distribution, making the perturbations visually imperceptible while retaining high attack strength. This enables a comprehensive examination of how forensic detectors respond to subtle or spatially constrained manipulations, while also enabling the generated images to bypass detection by appearing visually natural and free of noticeable adversarial noise.

Figure 2: How Diff-PGD works

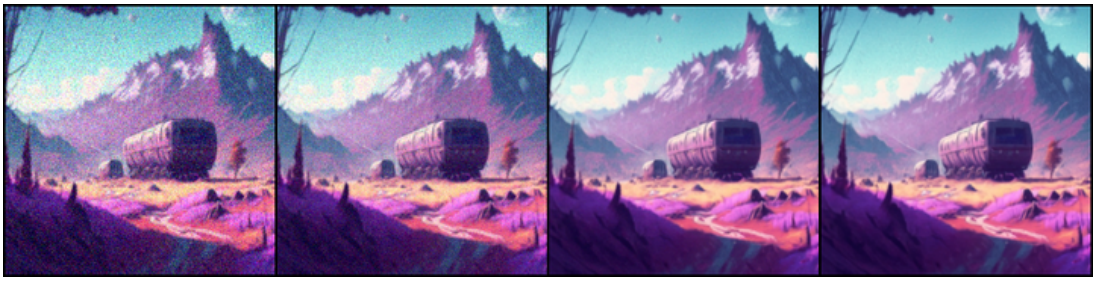
Require: Target classifier f_θ, original image x, mask M, denoiser D_ϕ, # of reverse SDEdit steps K, iterations n, stepsize η, clip value ϵ (when $M = 1$ it reduces to Diff-PGD)
$x^0 = x_0^0 = x$, $x_c = x$
for $t = 0, 1, 2, \ldots, n-1$ **do**
  $x_K^t \sim q(x_K^t | x^t)$ ▷ Sample x_K^t from $q(x_K^t | x^t)$ in each PGD iteration
  for $i = K-1, \ldots, 0$ **do**
    $x_i^t = R_\phi(x_{i+1}^t)$ ▷ Apply denoiser R_ϕ to x_{i+1}^t in each SDEdit iteration
    $x_i^t \sim M \circ x_i^t + (1-M) \circ q(x_i^t | x^t)$ ▷ Sample from masked combination of x_i^t and $q(x_i^t | x^t)$
  end for
  $g = \nabla_{x^t} l[f_\theta(M \circ x_0^t + (1-M) \circ x_c)]$ ▷ Compute the gradient
  $x^{t+1} = \Pi_{x,\epsilon}[x^t + \eta\, M \circ \text{sign}(g)]$ ▷ PGD update
end for
$x_0^n = \text{rSDEdit}(x^n)$ ▷ Apply reverse SDEdit to the final x^n
return x^n, x_0^n ▷ Return the final adversarial example and the denoised version

To simplify how Diff-PGD works:

1. Suppose the target detector is f. The goal is to find an adversarial image x that looks natural and causes f(x) to misclassify.
2. Traditional PGDs would just take the gradient of this loss with respect to the input image and do iterative updates to increase this loss, subjected to a constraint on total perturbation magnitude.
3. However, Diff-PGD uses a different method instead of relying solely on the adversarial loss gradient. It modifies this process by performing PGD-style optimisation within the diffusion model's latent space rather than directly in pixel space.
4. At each denoising step, Diff-PGD combines two gradients:
    a. Adversarial loss gradient – pushing the image to increase adversarial loss so the classifier would mis-predict more times, and
    b. Diffusion score gradient – pulling the image towards regions of high probability under the learned natural-image distribution, to make the image look more realistic.
5. Like PGD, Diff-PGD performs multiple update steps and would also ensure the perturbations do not exceed allowable boundaries.
6. The final adversarial image x is produced by the diffusion model's denoising process. It fools the target detector f while remaining close to the natural image manifold so the perturbation is stealthy, and the noise is visually unnoticeable to humans.

For a detailed process of noise generation in Diff-PGD, refer to Appendix 8.1.2. The figure below shows a sample of Diff-PGD's denoising process.

Figure 3: Denoising progression from undenoised to fully denoised state, from left to right



This optimisation of Diff-PGD provides a high level of controllability, since it produces visually natural perturbations that are difficult for both humans and forensic detectors to notice, while reliably inducing attacker-specified misclassifications. To summarise, it generates adversarial examples that look seemingly natural to humans while still achieving strong attack performance, hence making Diff-PGD a suitable choice for our experiments.

3.1.3 - Data Processing Development

Diff-PGD outputs three versions (original image, image with adversarial noise added, image with diffused adversarial noise) of each sample. These 3 versions are combined into a single 672-pixel width PNG image, thus we split this into separate pixel images using a Python script to allow for separation of outputs for ease of transfer to the detector pipeline. (Refer to Appendix 8.2.1) The following images are the result of this:

Figure 4.1: Pikachu_org.png

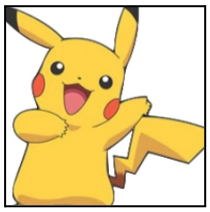


Figure 4.2: Pikachu_noise.png

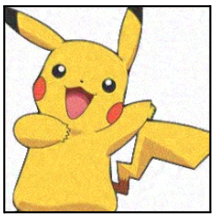
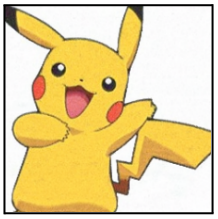


Figure 4.3: Pikachu_noisewithd.png



3.2 - Random noise

To provide a basis of comparison for a better evaluation of effectiveness of the adversarial models against AI detectors, we decided to create a separate test case where we used Python code to add random noise such as Gaussian noise or salt and pepper noise onto original images, and passed these into SPAI to determine changes in accuracy compared to the controlled dataset as well as adversarially attacked datasets. We added salt and pepper noise to images using Python code in Appendix 8.2.2, which adds a random number of pixels and places them at random pixels on the original image, each having a random grayscale value from 0 to 255. We added Gaussian noise to images using Python code in Appendix 8.2.3, which adds random, normally distributed values following a bell curve to a signal where small deviations are most likely and large ones are rare, creating random fluctuations that mimic natural interference in the image. Refer to Appendix 8.1.3 to understand how Gaussian Noise works.

Figure 5.1: Original



Figure 5.2: Salt and pepper



Figure 5.3: Gaussian



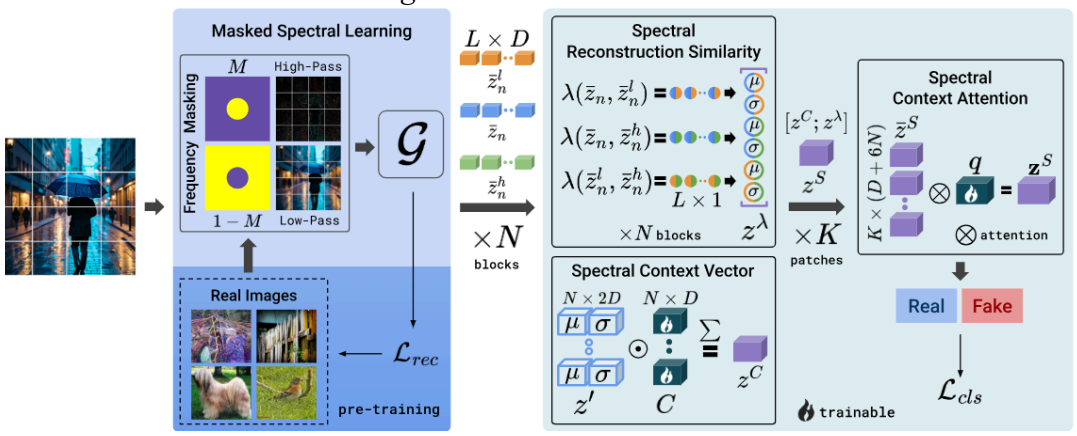
3.3 - AI Detection Approaches

To facilitate the process of discovering vulnerabilities in forensic AI image systems, we chose to use a typical system for AI detection of images by Generative Adversarial Networks (GANs) to generalize the types and impact of adversarial attacks that AI detectors could face. Typical AI detectors function by using various spectral image detection approaches where multiple techniques such as spectral pattern analysis, artifact detection and analysis of digital inconsistencies left by GANs.

3.3.1 - Spectral AI-Generated Image Detector (SPAI)

The detection method we used is Spectral AI-Generated Image Detector (SPAI) which employs spectral learning to learn the spectral distribution of real images under a self-supervised setup, then by using the spectral reconstruction similarity it detects AI-generated images as out-of-distribution samples of this learned model. [6]

Figure 6: How SPAI works



In detail, it establishes the spectral distribution of real images to serve as an invariant pattern for distinguishing between real and AI-generated images, then utilizes a self-supervised approach to train the model to learn the spectral distribution of real images by masking sections of the image and employing it with a pretext task of frequency reconstruction to teach the model spectral patterns of real images.

Thereafter, spectral reconstruction similarity (SRS) is used to measure the divergence from this learned distribution and detect AI-generated images as out-of-distribution samples of this model. SRS calculates the similarity between the features of the original image and its low/high-frequency components, expecting larger distances for AI-generated images compared to real ones.

At the same time, the use of spectral context vector captures the spectral context under which the spectral reconstruction similarity values are computed, while spectral context attention

(SCA), a mechanism that enables efficient processing of images at any resolution without prior pre-processing, allowing this approach to capture subtle spectral inconsistencies in high-resolution images. This is important as capturing subtle clues in images is crucial for effectively distinguishing between real and AI-generated content.

3.3.2 - User Interface (UI) Development

To simplify this detection tool, we developed this onto a webpage using Gradio by integrating both input photo and output results into one page using Python. We developed two different versions, (1) to evaluate a single photo and output certainty as well as whether it is AI-generated or real, (2) to evaluate accuracy of two directories pre-classified into a set of AI-generated photos and another set of real photos, then output metrics and attach a CSV file containing detailed results of the certainty score as evaluated by SPAI for each photo.
The Huggingface model can be found at:

- version (1): https://huggingface.co/spaces/jade-vine/SPAI-Detector-v1, and
- version (2): https://huggingface.co/spaces/jade-vine/SPAI-Detector-v2.

3.3.3 - Detection Metrics Development

To allow for efficient evaluation and comparison of results as produced by the detector when larger amounts of data are being evaluated, we decided to include a Python script (Refer to Appendix 8.2.4 for the script) to evaluate the metrics, including accuracy, precision, recall and F1 score of the model (See Appendix 8.1.4 for context), as well as including a confusion matrix to provide a summary of the results.

4. RESULTS

Adversarial Attacks were run on Windows 10 operating system, powered with an Intel Core i7-7820HQ CPU @ 2.90GHz, with 31.9GB of memory. The detection process was run on Windows 11 operating system, powered with an Intel Core i7-14700HX CPU, with 31.6GB of memory.

4.1 - Test With Controlled Dataset Without Adversarial Attacks

The metrics and confusion matrix of SPAI of 50 AI-generated and 50 real images without adversarial attacks sourced from Huggingface database [7] are as shown below:

Table 1.1: Metrics without attack

Metric	Value (3s.f.)
Accuracy	0.730
Precision	0.735
Recall	0.720
F1 Score	0.727

Table 1.2: Confusion matrix without attack

	Predicted pos	Predicted neg	Total
Actual pos	TP = 36	FN = 14	50
Actual neg	FP = 13	TN = 37	50

4.2 - Test with random noise

We first tested SPAI with random noise, namely Gaussian and salt and paper noise to all the control images in section 4.1, and passed the altered images into SPAI and obtained the metrics, which was same for both Gaussian and salt and pepper noise image datasets, a 0.5 accuracy where all AI-generated images were falsely classified as real and all real images

were classified correctly and 0 for precision, recall and f1 score. (TP = 0, FN = 50, FP = 0, TN = 50)

4.3 - Test with adversarial images from AnyAttack

We manipulated images by passing the original image and a target image through the AnyAttack model with the ability to change settings of both EPS and iteration steps to get a final adversarial image and tested its capabilities with the following tests.

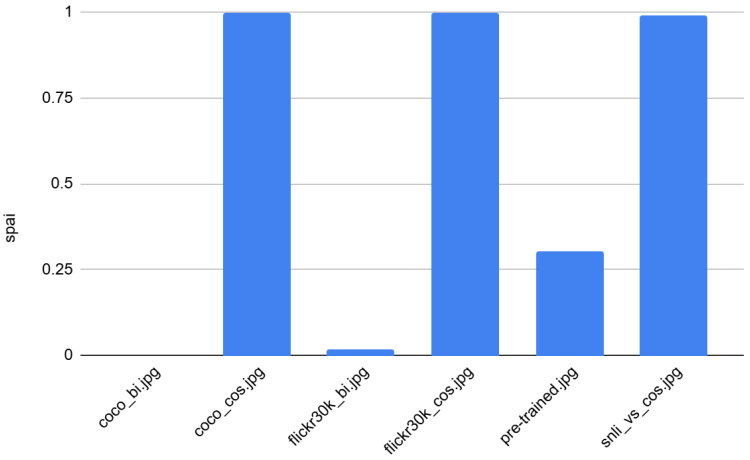
4.3.1 - Test with different models

We tested SPAI with various models of AnyAttack to see how the detector would perform against the same target and original image but altered across different model weights of AnyAttack. SPAI score is where 1 stands for 100% certainty of an AI-generated picture. Results are shown below: (Refer to Appendix 5 for a detailed version of the table)

Table 2: SPAI results of images generated by different models of AnyAttack at 20 EPS

Weight Model	SPAI score	Prediction
coco_bi	1.63×10^{-9}	Real
coco_cos	9.96×10^{-1}	AI
flickr30k_bi	1.70×10^{-2}	Real
flickr30k_cos	1.00	AI
pre-trained	3.02×10^{-1}	Real
snli_vs_cos	9.88×10^{-1}	AI

Figure 7: Bar Chart representing SPAI score of different models of AnyAttack



4.3.2 - Test at different EPS (ε)

We conducted a series of trial-and-error experiments and determined that the coco_bi weight model delivered the strongest performance, particularly in its ability to deceive forensic detectors even at low perturbation levels ε. Using this model, we generated an additional batch of adversarial images (we maintained the same target and original inputs) while varying ε from 10 to 18 with a single optimisation step for each setting, and passed them through the forensic detector SPAI. The resulting outputs are shown below in Table 3.1. (Refer to Appendix 6 for a detailed version of the table) We then decided to test with other pictures to see how the detector would handle a larger range of pictures to confirm our findings from previous results. We used a new set of target and original images to generate a separate batch of adversarial images with varying EPS from 18 to 30 with a single optimisation step for each setting, and passed them through the forensic detector SPAI. The resulting out are as shown below in Table 3.2(Refer to Appendix 7 for a detailed version of the table):

Table 3.1: Results of images at EPS 10 - 18

EPS value	SPAI score	Prediction
10	2.74×10^{-6}	Real
12	6.92×10^{-8}	Real

Table 3.2: Results of images at EPS 18 - 32

EPS value	SPAI score	Prediction
18	9.89×10^{-1}	AI
22	1.72×10^{-1}	Real

14	5.18×10^{-8}	Real
16	7.04×10^{-7}	Real
18	6.89×10^{-9}	Real

26	0.00	Real
30	1.20×10^{-10}	Real

4.4 - Test with adversarial images from Diff-PGD

As the noise seen in pictures generated by AnyAttack was very obvious, we wanted to find another method of incorporating noise into images that is undistinguishable by eye, thus we decided to test out Diff-PGD, of which the images generated with diffusion seem to be clean to the eye but contains enough noise to affect the accuracy of AI detectors. Refer to Figure 4.2 and 4.3 for examples of these images.

4.4.1 - Test with different EPS & iteration settings

We tested an input dataset of 40 different adversarial images at 10 iterative steps at 30 and 45 EPS with diffusion, generated by Diff-PGD, and passed them through SPAI to obtain the metrics. The following table shows the results that we obtained:

Table 4.1: Results with diffusion

EPS value	Iteration steps	Accuracy
20	20	0.579
35	20	0.684
35	5	0.667
30	10	0.529
45	10	0.618

Table 4.2: Results without diffusion

EPS value	Iteration steps	Accuracy
20	20	0.368
35	20	0.421
35	5	0.200
30	10	0.471
45	10	0.412

4.4.2 - Test with original controlled dataset

We used our previous controlled dataset of 50 AI-generated and 50 real images from section 4.1, and generated adversarial images through Diff-PGD with the optimal settings of 25 EPS with 15 optimisation steps that we concluded after previous tests, and passed them through SPAI to obtain the metrics. The following table shows the results that we obtained:

Table 5.1: Metrics with diffusion

Metric	Value
Accuracy	0.457
Precision	0.417
Recall	0.476
F1 Score	0.444
Confusion matrix	TP=24, FN=26, FP=33, TN=17

Table 5.2: Metrics without diffusion

Metric	Value
Accuracy	0.489
Precision	0.391
Recall	0.214
F1 Score	0.277
Confusion matrix	TP=11, FN=39, FP=13, TN=37

DISCUSSION

5.1 - Effect of random noise on detector

Random noise had a sizable effect on the accuracy of SPAI, as seen in the decrease in accuracy from 0.730 to 0.500. However, the degree of accuracy for images with random noise in each class (AI or Real) was very different compared to that of the control. With added random noise, SPAI classified every image to be real regardless of its original class. In other words, it failed to recognize any of the AI images while successfully classifying all the real images correctly. This shows that added random noise severely hinders the ability of SPAI to detect out-of-distribution images.

This agrees with previous findings, as past research has shown that random noise would hinder AI detectors due to addition of pixels that would hinder pattern recognition when these altered images are analyzed by them. Especially for SPAI, which utilizes SCA to identify spectral inconsistencies, would easily detect this noise, disrupting the structured spectral patterns necessary for SPAI to reliably detect AI images, causing it to predict images wrongly. Additionally, the added random noise breaks coherence in frequencies as detected in SPAI during analysis, potentially affecting pattern recognition in SRS, causing all of the images to be classified as real.

However, despite being able to cause SPAI to fail at recognizing all of the AI images, the noise in these images is very obvious, obscuring large areas of the image and causing them to lose much of the quality of the original image. This makes the attack impractical in real-world scenarios, where imperceptibility is essential.

5.2 - Effect of AnyAttack on detector

Different models of AnyAttack generated a vastly different outcome of SPAI score with the same input image. As seen in Table 2, model coco_bi generated the lowest score out of all the models, indicating the greatest certainty that the altered image is real. Changing the EPS of the coco_bi model, we found out that generally, as the EPS value increases, the SPAI score decreases, indicating better performance with greater EPS value. This is expected, as a greater EPS value of the model during generation of adversarial images increases the strength or magnitude of adversarial noise, decreasing the accuracy of the detector.

However, a greater EPS value can also increase the visibility of this noise in the image itself, as the noise from the target image becomes more visible. Due to the fact that AnyAttack produced images with moderately visible noise at any EPS value, it was not a practical method for adversarial attacks.

5.3 - Effect of Diff-PGD on detector

In the experiments using adversarial images at different EPS values and iteration settings, SPAI's accuracy was lower for images without diffusion compared to those with diffusion, ranges of 0.2 to 0.471 without diffusion and ranges of 0.529 to 0.684 respectively, indicating that images with raw adversarial noise were more detrimental to the accuracy of SPAI's spectral-based detection. This is expected, as diffusion smoothens out sharp pixels of adversarial noise, causing the frequency of those areas to be more constant rather than fluctuating with noise, and SRS would detect this as an indicator of a real image.

However, when the EPS value was increased in various test cases with constant iteration steps, the accuracy increased for images with and without diffusion, which contradicts

previous findings that EPS reduces accuracy. We hypothesize this to be due to the model pushing pixels with greater magnitude when EPS increases, causing them to enter a frequency-saturation region, which makes SRS and SCA in SPAI to become less sensitive, thus perturbations stop shifting spectral patterns in an adversarial direction.

Using optimised settings (25 EPS, 15 iterations) obtained through trial and error, SPAI's accuracy dropped significantly to 0.457 with diffusion, and 0.489 without diffusion. This is a large drop compared to the accuracy of the control dataset at 0.73 and that of the dataset with random noise at 0.5, showing that Diff-PGD is capable of confusing SPAI on both real and AI-generated images, a behaviour not observed with random noise or AnyAttack.

Looking at SPAI's confusion matrix, SPAI misclassified 26 out of 50 AI images as real and 33 out of 50 real images as AI with diffusion images, showing that Diff-PGD disrupts SPAI's learned spectral patterns rather effectively, causing it to lose its ability to distinguish the two classes reliably. Without diffusion, SPAI mostly failed at detecting AI images, showing that raw PGD-style perturbations more strongly disguise AI images as real, though they are visually more noticeable.

We believe that the effectiveness of Diff-PGD comes from its core design, which optimises the adversarial objective and constrains the perturbation to lie close to the natural image manifold using a diffusion model. Therefore, Diff-PGD is able to produce effects that align with the observations in Section 4.4.

5.4 - Limitations

Increasing strength of adversarial noise comes at a trade-off, while being able to efficiently affect accuracy of AI detectors, it can also visually distort images and would be unrealistic in a real-world scenario. In addition, as our study only used one kind of AI detector, the findings presented in our study could be different with other types of AI detectors with different frameworks.

5.5 - Conclusion and Future Work

This study examined the robustness of SPAI against different categories of adversarial perturbations using both controlled datasets and varied adversarial settings. Overall, the results demonstrate that SPAI, while effective under clean conditions, remains vulnerable to both traditional and modern adversarial manipulation. These findings underscore the challenges of developing robust AI-forensic systems, particularly as generative and adversarial techniques continue to evolve.

Future research can focus on several avenues to strengthen the resilience of AI-generated image detectors. A promising direction is the incorporation of multi-modal or hybrid detection strategies. Another potential direction is to train more detectors using adversarially augmented datasets, where models are exposed to a wide range of perturbations to improve robustness.

6. ACKNOWLEDGEMENTS

We would like to extend our sincere appreciation to the Defence Science and Technology Agency for offering the Research@YDSP programme. We would also like to thank Ms. Esther Chew Mei Ping for her support in managing the administrative aspects of the project. Finally, we express our heartfelt gratitude to Mr. Daniel Lee Jek Han for his mentorship and guidance throughout the project cycle.

7. REFERENCES

[1] Albert. (2025, May 30). AI Statistics In 2025: Key Trends And Usage Data. Digital Silk. https://www.digitalsilk.com/digital-trends/ai-statistics/

[2] Hawkingson, K. (2023, July). The most reliable AI image detectors can be tricked by simply adding texture to an image — a worrying find as AI disinformation plagues the internet and threatens political campaigns. https://www.businessinsider.com/ai-image-detectors-tricked-by-adding-texture-to-image-report-2023-7

[3] Pandit, B. (2024, August 29). Vision Language Models (VLMs) Explained. Datacamp.com; DataCamp. https://www.datacamp.com/blog/vlms-ai-vision-language-models

[4] Zhang, J., Ye, J., Ma, X., Li, Y., Yang, Y., Chen, Y., Sang, J., & Yeung, D.-Y. (2024). AnyAttack: Towards Large-scale Self-supervised Adversarial Attacks on Vision-language Models. ArXiv.org. https://arxiv.org/abs/2410.05346

[5] Xue, H., Araujo, A., Hu, B., & Chen, Y. (2023). Diffusion-Based Adversarial Sample Generation for Improved Stealthiness and Controllability. ArXiv.org. https://arxiv.org/abs/2305.16494

[6] Karageorgiou, D., Papadopoulos, S., Kompatsiaris, I., & Gavves, E. (2025) Any-Resolution AI-Generated Image Detection by Spectral Learning (SPAI). IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) https://mever-team.github.io/spai/

[7] Gurung H. (2024, Mar) AI-Generated vs Real Images Datasets. https://huggingface.co/datasets/Hemg/AI-Generated-vs-Real-Images-Datasets

8. APPENDIX

Appendix 8.1 - Definitions and Processes

8.1.1 - AnyAttack Adversarial Noise Generation

The process of generating adversarial noise in Anyattack can be summarised in the following equation:

$$min\, L(\, f_s(\delta\ + x_r\,), f_s(x)),\ s.t.\ x_r \neq\ x$$

Where:

- x is an original image from the large-scale dataset
- x_r is a random image, unrelated to x, used to receive the adversarial perturbation
- δ is the adversarial noise generated by the noise generator (decoder),
- $f_s(\cdot)$ is a frozen surrogate image encoder, used to embed images into a representation space,
- $L(\cdot, \cdot)$ is the similarity loss between embeddings, enforcing that the embedding of the image with adversarial noise, $x_r + \delta$, is close to that of x,
- $x_r \neq x$ is a constraint which ensures that we are not simply reconstructing the same image, instead the noise must "transport" a random image into the embedding of another image.

8.1.2 - Diff-PGD Processes

The process of generating adversarial noise in Diff-PGD can be summarised in the following equation:

$$x^{t+1} = P_{B(x,\,\epsilon)}[x^t\ +\ \ n\, sign \nabla_{x^t} l(\, f_\theta(x_0^t),\ y)\ ]$$

Where:

- n is the number of steps in the adversarial attack,
- y is the image x attacker-specified target label that the adversarial image should be classified as,
- sign ∇_{x^t} gives the direction to modify the image x^t to achieve the attack,
- $P(\cdot)$ is the projection operator that clips or projects its argument back into the allowed perturbation ball,
- $B(x, \epsilon)$ is the ball which enforces the perturbation constraint around the original image x by its maximum allowed perturbation ϵ,
- $f_\theta(\cdot)$ is the target classifier/detector, parameterized by θ to be attacked, and
- x^t is the adversarial image at iteration t.

$$\text{And:}\quad x_0^t = SDEdit(x^t,\ K)$$

Where:

- The parameter K can be used to control the edition-content preference.
- This means that a large K value brings more editions to the image and a small K value preserves more original content.

For context, the SDEdit (Stochastic Differential Editing) function is defined as:

$$SDEdit(x,\ K)\ =\ R_\phi(\dots R_\phi(R_\phi(x_K,\ K),\ K\ -\ 1)\dots\ , 0).$$

The key idea of SDEdit is:

1. to perform a forward diffusion step by diffusing the original distribution with a noisy intermediate state $x_K \sim q(x_K \mid x)$, adding noise to the image over K steps
2. apply a reverse denoising process R_ϕ parametrized by diffusion model weights ϕ in a Diffusion Model, to gradually reconstruct the image. This reverse trajectory guides the sample back toward the natural image manifold, producing an edited image that is both realistic and semantically coherent.
3. This process works as a bridge between the input distribution and the realistic data distribution, which can then be used in tasks such as stroke-based image synthesis and editing, and adversarial samples purification.

The process of denoising adversarial images in Diff-PGD can be summarised in the following equation:

$$p(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\phi(x_{t-1} \mid x_t)$$

Where:

- T is the total number of diffusion steps used during adversarial noise generation process, and therefore, denoising process,
- ϕ represents the parameters of the diffusion model,
- x_0 is the final denoised image produced after the full reverse diffusion process,
- x_T is the fully noised image generated from the first equation,
- x_t is the image at step t in the denoising process, and
- $p(\cdot)$ is the probability distribution of the noised image.

8.1.3 - Gaussian Noise

The exact formula of how Gaussian noise is produced is:

$$p(z) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

Where:

- p(z) is the probability density of observing a noise value z in the Gaussian distribution,
- σ is the standard deviation of the noise distribution and determines the intensity of noise fluctuations, and
- μ is the mean of the Gaussian distribution, which determines the average value of the noise. The following show images with different random added noise.

8.1.4 - Accuracy, Precision, Recall, F1 Score

The definitions of Accuracy, Precision, Recall and F1 Score is shown in the following equations:

$$A = \frac{TP+TN}{TP+TN+FP+FN},\ P = \frac{TP}{TP+FP},\ R = \frac{TP}{TP+FN}, \text{ and } F1 = \frac{2PR}{P+R}$$

Where:

- A, P, R, F1 stands for Accuracy, Precision, Recall and F1 Score respectively,
- TP, TN, FP, FN represents “True Positive”, “True Negative”, “False Positive” and “False Negative” respectively,

And:

- True Positive (correct prediction) means that the model predicts an AI-generated image as AI-generated,

- True Negative (correct prediction) means that the model predicts a real image as real,
- False Positive (incorrect prediction) means that the model predicts a real image as AI-generated,
- False Negative (incorrect prediction) means that the model predicts an AI-generated image as real.

Appendix 8.2 - Algorithms and Codes

8.2.1 - Code for data processing

```python
import os
from PIL import Image

def split_three_pngs(input_folder, output_folder):
    os.makedirs(output_folder, exist_ok=True)

    for filename in os.listdir(input_folder):
        if not filename.lower().endswith(".png"): # only process PNG images
            continue
        img_path = os.path.join(input_folder, filename)
        img = Image.open(img_path)

        w, h = img.size
        if w != 672: # only process width 672 images
            continue

        left_box   = (0, 0, 224, h)
        mid_box    = (224, 0, 448, h)
        right_box  = (448, 0, 672, h)

        left_img  = img.crop(left_box)
        mid_img   = img.crop(mid_box)
        right_img = img.crop(right_box)

        base = os.path.splitext(filename)[0]

        left_img.save(os.path.join(output_folder, f"{base}_org.png"))
        mid_img.save(os.path.join(output_folder, f"{base}_noise.png"))
        right_img.save(os.path.join(output_folder, f"{base}_noisewithd.png"))

        print(f"Processed {filename}")

    print("All completed")

```

8.2.2 - Code for salt and pepper noise

```python
import random
import cv2
import os

def add_noise(img):

    row, col = img.shape

    number_of_pixels = random.randint(300, 10000)
    for i in range(number_of_pixels):
        y = random.randint(0, row - 1)
        x = random.randint(0, col - 1)
        img[y, x] = 255

    number_of_pixels = random.randint(300, 10000)
    for i in range(number_of_pixels):
        y = random.randint(0, row - 1)
        x = random.randint(0, col - 1)
        img[y, x] = 0

    return img


src_dir = r"aifolder"
for img_file in os.listdir(src_dir):

    full_path = os.path.join(src_dir, img_file)

    img = cv2.imread(full_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print("Skipping non-image:", img_file)
        continue

    # Output file with unique name (avoid overwriting)
    out_name = f"noise_{img_file}"
    output_path = os.path.join(src_dir, out_name)

    cv2.imwrite(output_path, add_noise(img))
```

8.2.3 - Code for Gaussian noise

```python
import os
import cv2
import numpy as np
import random

def gaussian_noise(img, mean, stddev, gamma=1):
    # Create Gaussian Noise
    gauss_noise = np.zeros(img.shape[:2], dtype=np.float32)
    cv2.randn(gauss_noise, mean, stddev)  # fill with Gaussian noise
    gauss_noise = (gauss_noise * gamma).astype(np.uint8)

    # If grayscale
    if len(img.shape) == 2:
        output = cv2.add(img, gauss_noise)

    # If color
    elif len(img.shape) == 3:
        merged = cv2.merge([gauss_noise, gauss_noise, gauss_noise])
        output = cv2.add(img, merged)

    return gauss_noise, output

src_dir = r"aifolder"
for img_file in os.listdir(src_dir):
    full_path = os.path.join(src_dir, img_file)
    img = cv2.imread(full_path)

    if img is None:
        print("Skipping non-image:", img_file)
        continue

    out_name = f"noise_{img_file}"
    output_path = os.path.join(src_dir, out_name)
    noise, noisy_img = gaussian_noise(img, 5, random.randint(0,255), gamma=1)
    cv2.imwrite(output_path, noisy_img)
```

8.2.4 - Code for metrics calculation

```python
# 7. Compute metrics
TP = ((df["pred"] == 1) & (dx["class"] == 1)).sum()
TN = ((df["pred"] == 0) & (dx["class"] == 0)).sum()
FP = ((df["pred"] == 1) & (dx["class"] == 0)).sum()
FN = ((df["pred"] == 0) & (dx["class"] == 1)).sum()

total = len(df)
acc = (TP + TN) / total if total else 0
prec = TP / (TP + FP) if (TP + FP) else 0
rec = TP / (TP + FN) if (TP + FN) else 0
f1 = (2 * prec * rec) / (prec + rec) if (prec + rec) else 0


df["results"] = df["pred"].map({1: "AI", 0: "Real"})
df["original"] = dx["class"]
resultdict = {fnamelist[i]: label for i, label in enumerate(df["results"])}
RESULTS_EXPORT = os.path.join(OUTPUT_DIR, "input1.csv")
df.to_csv(RESULTS_EXPORT, index=False)


#8. Return summary text
return (
    f"Samples: {total}\n"
    f"Accuracy:  {acc:.3f}\n"
    f"Precision: {prec:.3f}\n"
    f"Recall:    {rec:.3f}\n"
    f"F1 Score:  {f1:.3f}\n\n"
    f"Confusion Matrix\n"
    f"TP: {TP}, FP: {FP}\n"
    f"FN: {FN}, TN: {TN}\n\n"
    f"Full results\n"
    f"Image {resultdict}",
    RESULTS_EXPORT
)
```

Appendix 8.3 - Detailed Tables

8.3.1 - AnyAttack varied weight models testing

Weight Model	Class	SPAI Score	Image
Original image	0	0	
Target image	0	1.76E-07	
coco_bi	0	1.63E-09	
coco_cos	1	0.996529	
flickr30k_bi	0	0.017018	
flickr30k_cos	1	0.999544	
pre-trained	0	0.301601	
snli_vs_cos	1	0.988332	

Appendix 8.3.2 - AnyAttack EPS testing (10 to 18)

EPS	Class	SPAI Score	Image

10	0	2.74E-06	
12	0	6.92E-08	
14	0	5.18E-08	
16	0	7.04E-07	
18	0	6.89E-09	

Appendix 8.3.3 - AnyAttack EPS testing (New images, 18 to 30)

EPS	Class	SPAI Score	Image
Original Image	0	6.25E-26	
Target Image	0	0	
18	1	0.989	

22	0	0.172	
26	0	0	
30	0	1.20E-10	